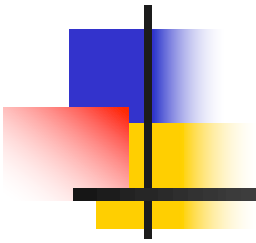
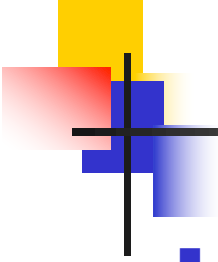


Tema 4: Capa Servicios REST con Spring



- Introducción
- Desarrollo de controladores
- Validación básica de datos
- Gestión de excepciones
- Internacionalización de mensajes
- Autenticación, autorización y control de acceso

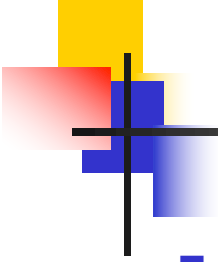


Introducción

- Usaremos la librería spring-web (incluida transitivamente por spring-boot-starter-web) para implementar ágilmente la capa Servicios REST
- Al usar Spring Boot, el servicio debe contener una clase anotada con `@SpringBootApplication`, que debe contener un método `main`

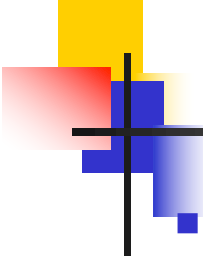
```
package es.udc.pashop.backend;

import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;
...
@SpringBootApplication
public class Application {
    ...
    public static void main(String[] args) {
        SpringApplication.run(Application.class, args);
    }
    ...
}
```



Introducción

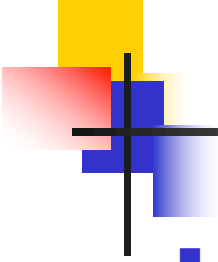
- Ejecución en desarrollo
 - `mvn spring-boot:run`
 - Localiza la clase anotada con `@SpringBootApplication` e invoca el método `main`
 - Método `main`
 - Arranca automáticamente un servidor de aplicaciones embebido
 - El contenedor de objetos de Spring escanea los `.class` para encontrar las clases de tipo bean (e.g. las anotadas con `@Service`, `@Repository`, `@RestController`, etc.) y crea los beans correspondientes
 - El backend está listo para recibir peticiones



Introducción

Ejecución en producción

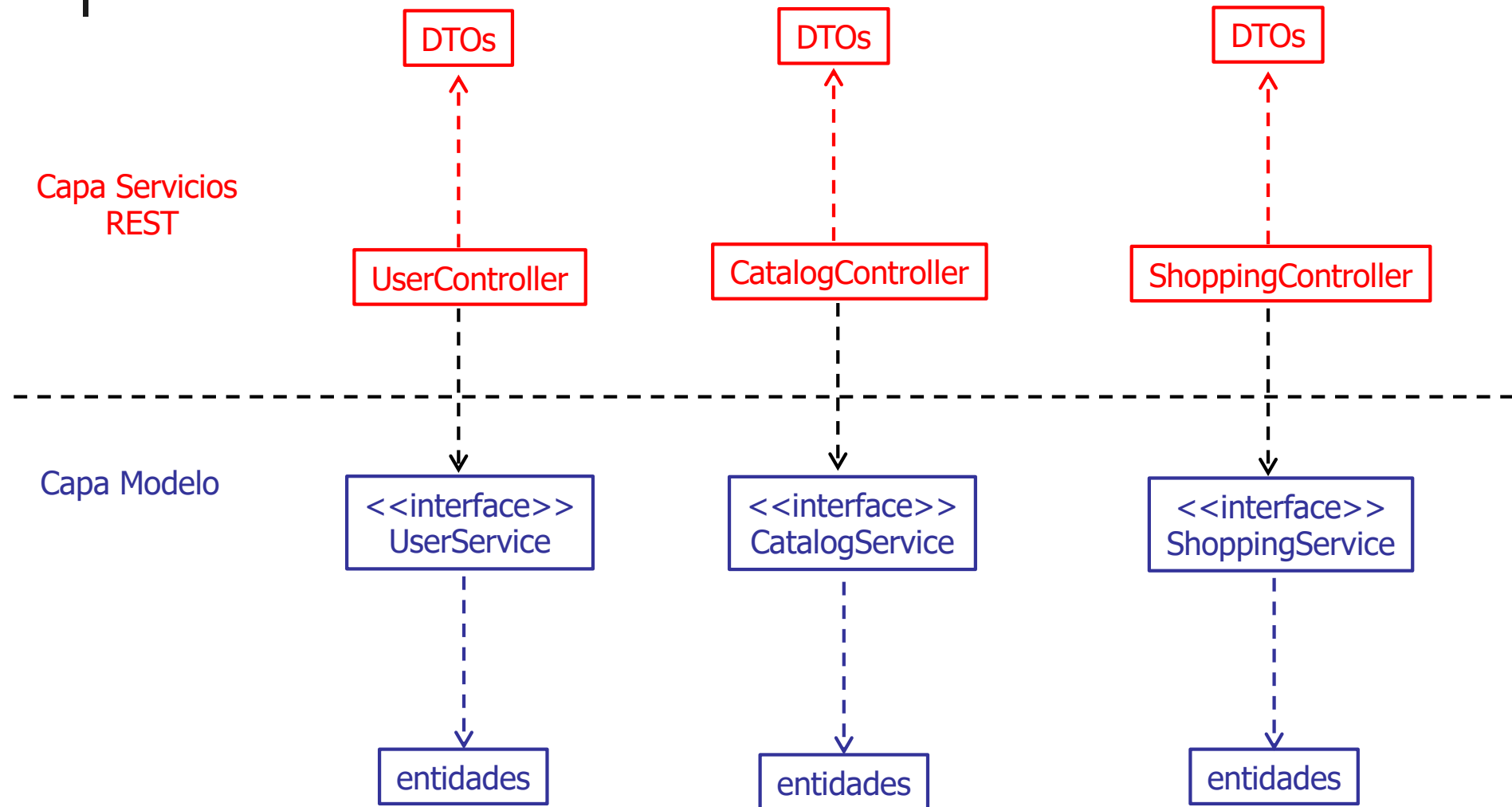
- **mvn package** El plugin `spring-boot` causa que en la fase `package` se genere el fichero `jar` con los contenidos que se comentan a continuación
- Opción 1
 - `java -jar target/pa-shop-backend.jar`
 - `pa-shop-backend.jar` es un “**fat JAR**” estándar, que contiene
 - Todos los `.class` del backend
 - Todas las librerías (ficheros `.jar`) de las que depende (inclusive un servidor de aplicaciones embebido)
 - Un fichero de meta-información (`META-INF/MANIFEST.MF`) que especifica el nombre completo de la clase anotada con `@SpringBootApplication`
 - La máquina virtual de Java ejecutará el método `main` de esa clase
- Opción 2
 - Para que la fase `package` genere un WAR válido es necesario seguir los pasos que indican en <https://github.com/spring-attic/gs-convert-jar-to-war>
 - Instalar `target/pa-shop-backend.war` en un servidor de aplicaciones Java estándar
 - Incluye lo necesario para que se inicialice el contenedor de objetos de Spring y esté listo para recibir peticiones

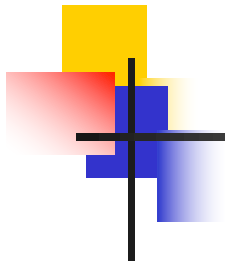


Desarrollo de controladores - Enfoque

- Definiremos una **clase controlador** por cada servicio local
- Conceptualmente, los controladores son similares a los servlets que desarrollábamos en ISD
 - Reciben las peticiones HTTP y delegan en un método de un servicio local
 - Internamente spring-web usa la API de Servlets
- Pero el nivel de abstracción es mucho mayor
 - **Un controlador tiene tantos métodos públicos como el servicio local asociado, con una firma similar**, aunque con diferencias (e.g. se usan DTOs en lugar de entidades)
 - Sobre los métodos de los controladores **se usan anotaciones para especificar cómo mapear las peticiones HTTP**
 - La conversión desde/a la representación de un recurso (JSON) a/desde un DTO será automática

Desarrollo de controladores





Desarrollo de controladores – UserController

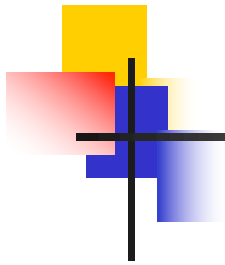
Recurso	Método	Entrada	Salida
/users/signUp ¹	POST	UserDto [body]	AuthenticatedUserDto ² [body]
/users/login ¹	POST	LoginParamsDto [body]	AuthenticatedUserDto ² [body]
/users/loginFromServiceToken ^{1,3}	POST	userId [jwt] ⁴	AuthenticatedUserDto ² [body]
/users/{id}	PUT	userId [jwt] ⁴ id [path], UserDto [body]	UserDto [body]
/users/{id}/changePassword ¹	POST	userId [jwt] ⁴ id [path], ChangePasswordParamsDto [body]	

[1] Se usa la técnica de overloaded POST porque no corresponden a operaciones CRUD

[2] **AuthenticatedUserDto** incluye los datos de perfil del usuario (excepto la contraseña) y la representación del carrito (para que el frontend cachee ambas cosas)

[3] **/users/loginFromServiceToken** se explica como parte del tema 7

[4] **userId** se pasa de forma segura mediante un JSON Web Token (JWT)



Desarrollo de controladores – CatalogController

Recurso	Método	Entrada	Salida
/catalog/categories	GET		List<CategoryDto> [body]
/catalog/products/{id}	GET	id [path]	ProductDto [body]
/catalog/products	GET	categoryId [param], keywords [param] , page [param]	BlockDto<Product- SummaryDto> [body]

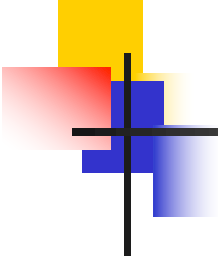
Desarrollo de controladores – ShoppingController

Recurso	Método	Entrada	Salida
/shopping/shoppingcards/{shoppingCardId}/addToShoppingCart ¹	POST	userId [jwt] ³ , shoppingCardId [path], AddToShoppingCartParamsDto [body]	ShoppingCardDto ² [body]
/shopping/shoppingcards/{shoppingCardId}/updateShoppingCartItemQuantity ¹	POST	userId [jwt] ³ , shoppingCardId [path], UpdateShoppingCartItemQuantityParamsDto [body]	ShoppingCardDto ² [body]
/shopping/shoppingcards/{shoppingCardId}/removeShoppingCartItem ¹	POST	userId [jwt] ³ , shoppingCardId [path], RemoveShoppingCartItemParamsDto [body]	ShoppingCardDto ² [body]
/shopping/shoppingcards/{shoppingCardId}/buy ¹	POST	userId [jwt] ³ , shoppingCardId [path], BuyParamsDto [body]	Long [body]
/shopping/orders/{orderId}	GET	userId [jwt] ³ , orderId [path]	OrderDto [body]
/shopping/orders	GET	userId [jwt] ³ , page [param]	BlockDto<OrderSummaryDto> [body]

[1] Se usa la técnica de overloaded POST porque no corresponden a operaciones CRUD

[2] Devuelven la nueva representación del carrito para que el frontend la cachee

[3] `userId` se pasa de forma segura mediante un JSON Web Token (JWT)



Desarrollo de controladores – Ejemplo 1

[Back](#)

Purchase order 14

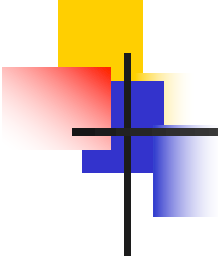
6/3/2025 - 1:48 PM

Rúe del Percebe, 13 - 12345

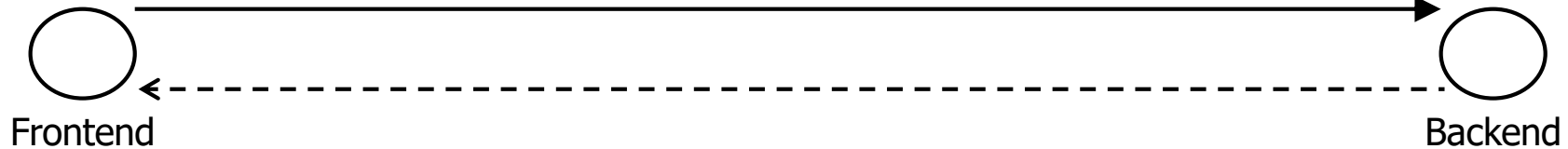
	Price	Quantity
2001: A Space Odyssey [Blu-ray]	€11.99	1
Blade Runner [Blu-ray]	€11.99	1
Unforgiven [DVD]	€7.65	1

Total price: €31.63

Desarrollo de controladores – Ejemplo 1



```
GET /shopping/orders/14 HTTP/1.1
...
```



```
HTTP/1.1 200 OK
...
{
  "id": 14,
  "items": [
    {
      "id": 24,
      "productId": 1,
      "productName": "2001: A Space Odyssey [Blu-ray]",
      "productPrice": 11.99,
      "quantity": 1
    },
    ...
  ],
  "date": "2025-06-03T13:48:50",
  "totalPrice": 31.63,
  "postalAddress": "Rue del Percebe, 13",
  "postalCode": "12345"
}
```

Desarrollo de controladores – Ejemplo 1

```
@RestController
@RequestMapping("/shopping")
public class ShoppingController {
```

```
...
```

```
@Autowired
```

```
private ShoppingService shoppingService;
```

```
...
```

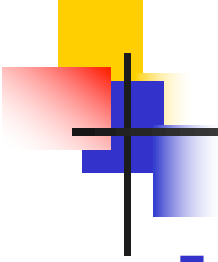
GET /shopping/orders/14

```
@GetMapping("/orders/{orderId}")
public OrderDto findOrder(@RequestAttribute Long userId,
    @PathVariable Long orderId)
    throws InstanceNotFoundException, PermissionException {
    return toOrderDto(shoppingService.findOrder(userId, orderId));
}
```

```
...
```

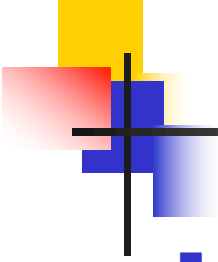
```
}
```

→ Conversión automática OrderDto → JSON en el cuerpo de la respuesta



Desarrollo de controladores – Ejemplo 1

- Excepto que se diga lo contrario, las anotaciones de mapping de peticiones HTTP (spring-web) pertenecen al paquete `org.springframework.web.bind.annotation`
- **@RestController**
 - Se usará para anotar cada controlador
 - Al igual que **@Repository** y **@Service**, sus instancias (beans) también son gestionadas por el contenedor de objetos de Spring
 - En consecuencia, podemos usar **@Autowired** para inyectarle el servicio local asociado



Desarrollo de controladores – Ejemplo 1

- Mapping de peticiones HTTP

- **@RequestMapping**

- La usaremos a nivel de controlador para especificar que las peticiones dirigidas a un path que comience por el valor especificado sean procesadas por este controlador

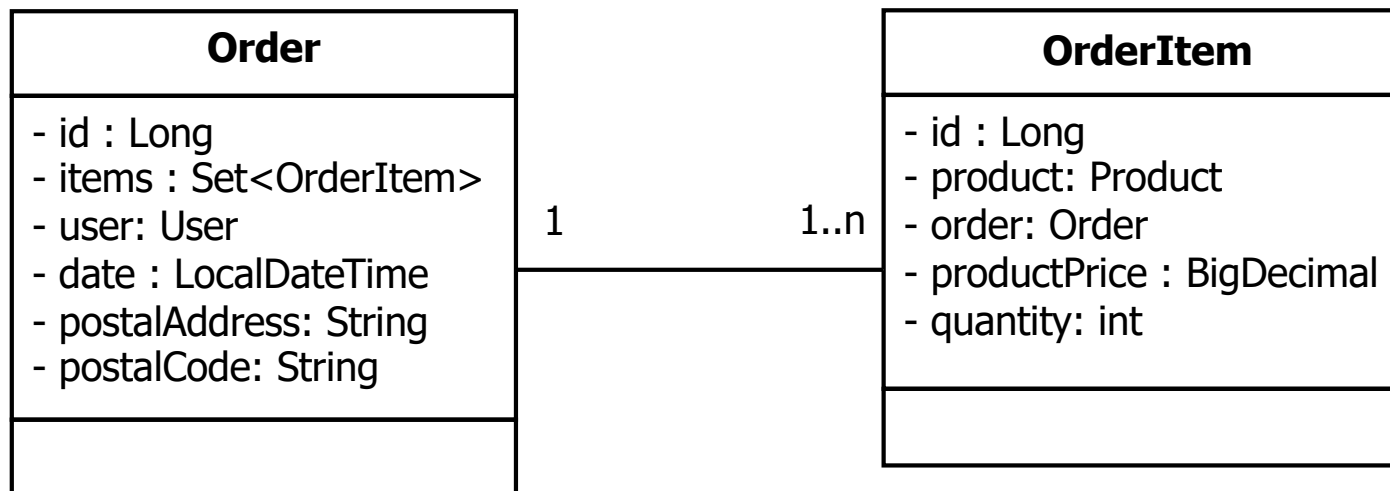
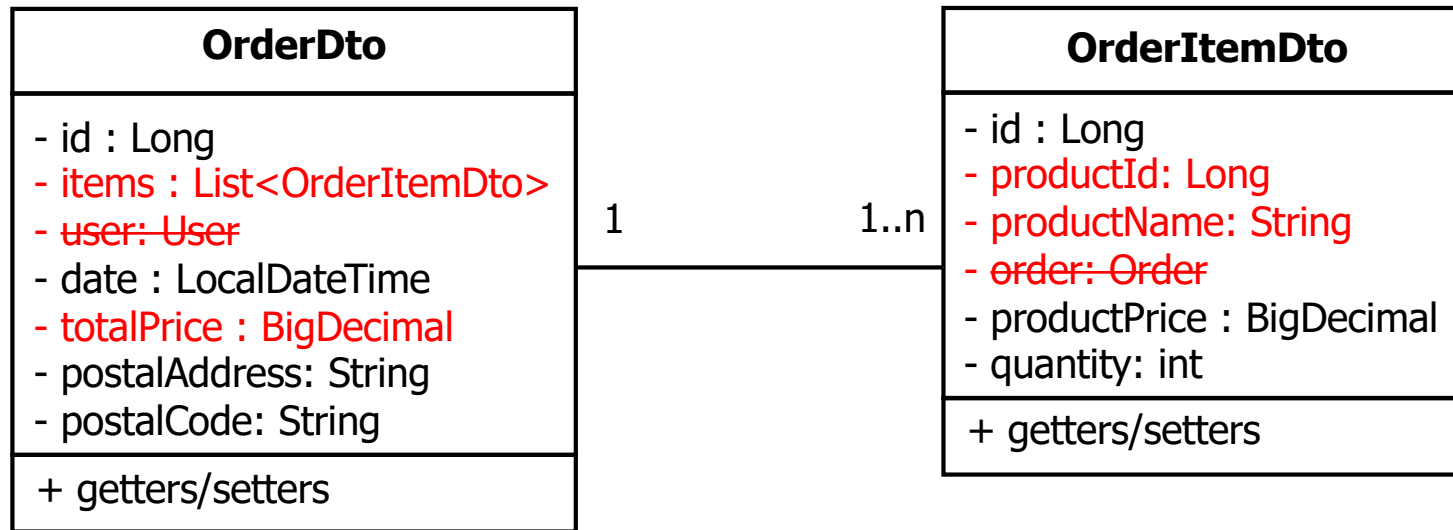
- **@{Get, Post, Put, Delete}Mapping**

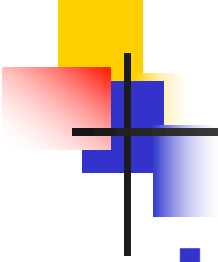
- Las usaremos a nivel de operación para especificar el método HTTP y el resto del path de la petición que causará la ejecución de esta operación
 - Se pueden usar expresiones del tipo `{variable}` para capturar el valor de una parte del path
 - En el ejemplo anterior se usa `/orders/{orderId}`

- **@PathVariable**

- Permite inyectar el valor de `{variable}` (path) en un parámetro del método (que tiene que ser de un tipo básico o `String`)
 - Por defecto, variable y parámetro tienen que tener el mismo nombre
 - En el ejemplo anterior, si la petición es `GET`
`http://.../shopping/orders/14`, el parámetro `orderId` de `findOrder` tomará el valor 14

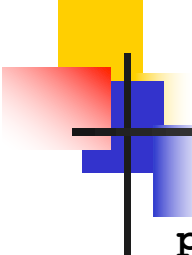
Desarrollo de controladores – Ejemplo 1





Desarrollo de controladores – Ejemplo 1

- Conversión automática desde/a JSON a/desde un DTO
 - spring-web usa Jackson: <https://github.com/FasterXML/jackson>
 - Los nombres de los campos JSON corresponden a los nombres de las propiedades (métodos `get/set`)
 - Incluye soporte para numerosos tipos Java (tipos primitivos, contrapartidas objetuales, `String`, colecciones, fechas, etc.)



Desarrollo de controladores – Ejemplo 1

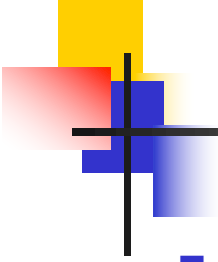
```
public class OrderConvensor {
    ...
    public final static OrderDto toOrderDto(Order order) {

        List<OrderItemDto> items = order.getItems().stream().map(
            i -> toOrderItemDto(i)).collect(Collectors.toList());

        items.sort(Comparator.comparing(OrderItemDto::getProductName));

        return new OrderDto(order.getId(), items,
            order.getDate(), order.getTotalPrice(),
            order.getPostalAddress(), order.getPostalCode());
    }

    private final static OrderItemDto toOrderItemDto(OrderItem item) {
        return new OrderItemDto(item.getId(),
            item.getProduct().getId(), item.getProduct().getName(),
            item.getProductPrice(), item.getQuantity());
    }
    ...
}
```



Desarrollo de controladores – Ejemplo 1

- Observación: Domain Model

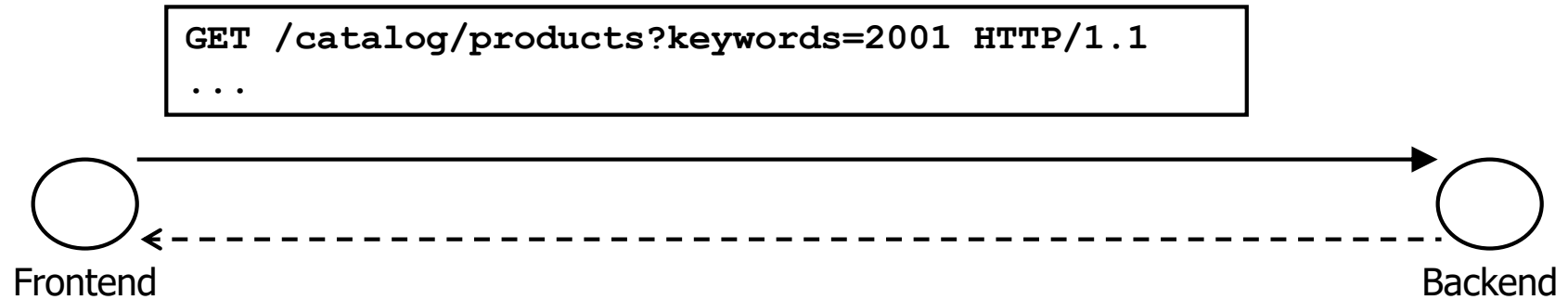
- `Order.getTotalPrice` simplifica y hace más natural la conversión de `Order` a `OrderDto`
- En `Order`

```
@Transient
public BigDecimal getTotalPrice() {
    return items.stream().map(i -> i.getTotalPrice()).reduce(
        new BigDecimal(0), (a, b) -> a.add(b));
}
```

- En `OrderItem`

```
@Transient
public BigDecimal getTotalPrice() {
    return productPrice.multiply(new BigDecimal(quantity));
}
```

Desarrollo de controladores – Ejemplo 2



```
HTTP/1.1 200 OK
...
{
  "items": [
    {
      "id": 1,
      "name": "2001: A Space Odyssey [Blu-ray]",
      "categoryId": 1
    }
  ],
  "existMoreItems": false
}
```

Desarrollo de controladores – Ejemplo 2

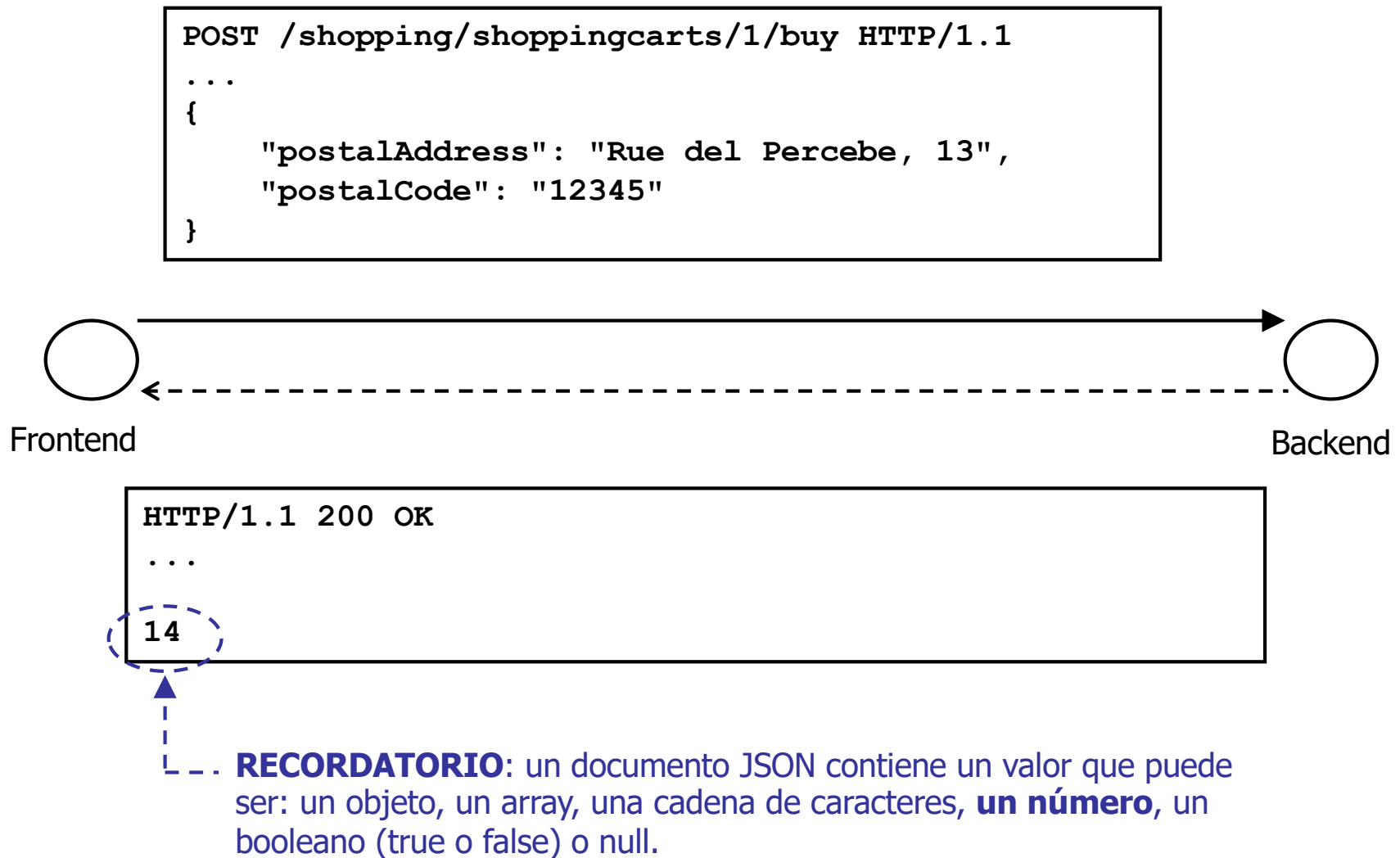
`@GetMapping("/products")`

GET /catalog/products?keywords=2001

```
public BlockDto<ProductSummaryDto> findProducts(  
    @RequestParam(required=false) Long categoryId,  
    @RequestParam(required=false) String keywords,  
    @RequestParam(defaultValue="0") int page) {  
  
    Block<Product> productBlock = catalogService.findProducts(categoryId,  
        keywords != null ? keywords.trim() : null, page, 10);  
  
    return new BlockDto<>(toProductSummaryDtos(  
        productBlock.getItems()),  
        productBlock.getExistMoreItems());  
}
```

- `@RequestParam` permite inyectar el valor de un parámetro HTTP en un parámetro del método (por defecto, deben tener el mismo nombre). El tipo Java del parámetro tiene que ser básico o `String`
- Por defecto, los parámetros se consideran obligatorios (por defecto, `required` es `true`)
- Si un parámetro es opcional, debe indicarse `required=false` (si el parámetro HTTP no se envía, el parámetro del método toma el valor `null`) o `defaultValue` (si el parámetro HTTP no se envía, el parámetro del método toma el valor especificado)
- En el ejemplo anterior: `categoryId=null, keywords=2001, page=0`

Desarrollo de controladores – Ejemplo 3



Desarrollo de controladores – Ejemplo 3

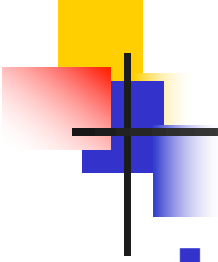
POST /shopping/shoppingcarts/14/buy

```
@PostMapping("/shoppingcarts/{shoppingCartId}/buy")
public Long buy(@RequestAttribute Long userId,
                @PathVariable Long shoppingCartId,
                @Validated @RequestBody BuyParamsDto params)
    throws InstanceNotFoundException, PermissionException,
           EmptyShoppingCartException {

    return shoppingService.buy(userId, shoppingCartId,
                               params.getPostalAddress(), params.getPostalCode()).getId();
}
```

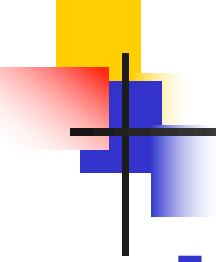
...

@RequestBody permite inyectar la representación contenida en el cuerpo de la petición en el parámetro anotado (conversión automática JSON → **BuyParamsDto**)



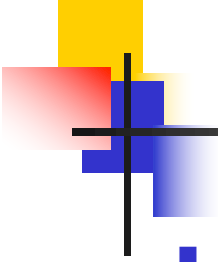
Validación básica de datos

- Desde la capa de Servicios REST, usaremos el soporte de spring-web y la API estándar de validación de Java (enfoque declarativo) para realizar validaciones básicas de los datos recibidos en las peticiones HTTP
 - Comprobar si un parámetro/campo obligatorio se envía
 - Comprobar que un entero pasado como un String es convertible a entero
 - Comprobar si un valor entero está dentro del rango permitido
 - Comprobar si el número de caracteres de un String está dentro de un rango permitido
 - Etc.
- En la mayor parte de los casos, la implementación de los casos de uso en la capa Lógica de Negocio no tendrá que encargarse de este tipo de validaciones básicas
 - El código queda más sencillo
 - Y no es un problema, dado que la capa Modelo sólo es accesible a través de la capa de Servicios REST



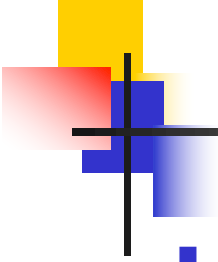
Validación básica de datos

- Los datos que se reciben en las peticiones HTTP provienen fundamentalmente de
 - {variable}s en el path
 - Parámetros HTTP
 - Representaciones en el cuerpo de la petición



Validación básica de datos

- spring-web valida automáticamente la obligatoriedad y la conversión de los valores recibidos en las {variable}s insertadas en el path y los parámetros HTTP, teniendo en cuenta los tipos Java de los parámetros correspondientes en el método al que mapea la petición
 - E.g. Si se invoca a POST
`http://.../shopping/shoppingcarts/abc/buy`, spring-web devuelve un 400 porque `abc` no es un valor convertible al tipo (`Long`) del parámetro `shoppingCartId` en `ShoppingController.buy`
 - Si la conversión no se puede hacer o un parámetro HTTP obligatorio no está presente => devuelve un 400 e incluye un JSON con información adecuada para depuración, pero no para presentar al usuario final
 - En situaciones similares a ésta no se genera ningún problema de usabilidad, dado que el frontend siempre pasará un valor correcto en el path (el usuario no teclea el `shoppingCartId`)
 - **Ejercicio: pensar si `CatalogController.findProducts` puede recibir valores incorrectos**



Validación básica de datos

- Sin embargo, existen situaciones en las que se desean realizar validaciones más finas (e.g. comprobar si un valor entero está dentro del rango permitido, comprobar si el número de caracteres de un `String` está dentro de un rango permitido, etc)
- Normalmente, este tipo de situaciones ocurre en peticiones POST/PUT
 - Registrar un usuario, actualizar los datos de perfil de un usuario, añadir un producto al carrito, realizar una compra, etc.
 - En todas estas situaciones, el usuario puede no introducir algún valor en un campo obligatorio, introducir una cadena de caracteres demasiado grande o pequeña, un valor numérico fuera de rango, etc.
- En este tipo de situaciones, pasaremos los datos como una representación en el cuerpo de la petición
 - La representación se mapeará a un DTO y sobre él especificaremos reglas de validación mediante anotaciones de la API estándar **Bean Validation**
 - <https://jakarta.ee/specifications/bean-validation/3.0>
 - <https://jakarta.ee/specifications/bean-validation/3.0/apidocs>
 - Spring incluye por defecto una implementación de esta API

Validación básica de datos – Bean Validation – Ejemplo

• Postal code: size must be between 1 and 20

Buy

Postal address

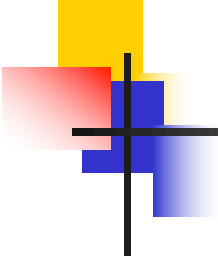
Rúe del Percebe, 13

Postal code

123456789012345678901

Buy

	Price	Quantity
2001: A Space Odyssey [Blu-ray]	€11.99	1
Blade Runner [Blu-ray]	€11.99	1
Unforgiven [DVD]	€7.65	1
Total price: €31.63		



Validación básica de datos – Bean Validation – Ejemplo

```
import org.springframework.validation.annotation.Validated;
```

```
...
```

```
@PostMapping("/shoppingcards/{shoppingCartId}/buy")
```

```
public Long buy(@RequestAttribute Long userId,
```

```
    @PathVariable Long shoppingCartId,
```

```
    < @Validated > @RequestBody BuyParamsDto params)
```

```
    throws InstanceNotFoundException, PermissionException,
```

```
        EmptyShoppingCartException {
```

```
    ...
```

```
}
```

- `@Validated` causa que spring-web valide los datos del DTO con la API de Bean Validation
- Si una o más reglas de validación se incumplen, Spring lanza la excepción de runtime `MethodArgumentNotValidException`

Validación básica de datos – Bean Validation – Ejemplo

```
import jakarta.validation.constraints.NotNull;  
import jakarta.validation.constraints.Size;
```

```
public class BuyParamsDto {
```

```
    private String postalAddress;
```

```
    private String postalCode;
```

```
    @NotNull
```

```
    @Size(min=1, max=200)
```

```
    public String getPostalAddress() {
```

```
        return postalAddress;
```

```
    }
```

```
    @NotNull
```

```
    @Size(min=1, max=20)
```

```
    public String getPostalCode() {
```

```
        return postalCode;
```

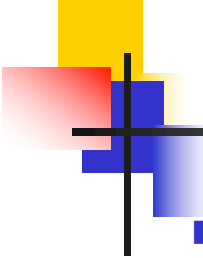
```
    }
```

```
    // Métodos set ...
```

```
}
```

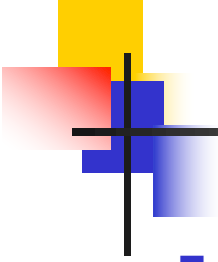
Reglas de validación (se anotan los atributos XOR los métodos `get` de las propiedades)

- Para que spring-web pueda ejecutar las reglas de validación, se necesita definir un bean que implemente `jakarta.validation.Validator`
- Ver código fuente de `es.udc.pashop.backend.Application`



Gestión de excepciones

- spring-web ofrece varios mecanismos para que un método de un controlador pueda devolver un código de respuesta y una representación en el cuerpo de la respuesta
- Por defecto, cuando la ejecución de un método de un controlador no devuelve ninguna excepción, se devuelve
 - Un código de respuesta 200 (OK)
 - La representación del valor retorno en el cuerpo de la respuesta
- Cuando la ejecución de un método de un controlador puede devolver una excepción, spring-web permite
 - Indicar el código de la respuesta
 - Devolver información sobre el error en el cuerpo de la respuesta



Gestión de excepciones

- En el caso de estudio, en la implementación de los métodos de los controladores no se tratan las excepciones devueltas por los métodos de los servicios locales
 - Se declaran en la cláusula **throws** de los métodos de los controladores
 - Se usan anotaciones para especificar cómo tratar cada excepción (código de respuesta + contenido de la respuesta)

Gestión de excepciones – Ejemplo 1

```
POST /shopping/shoppingcarts/1/buy
```

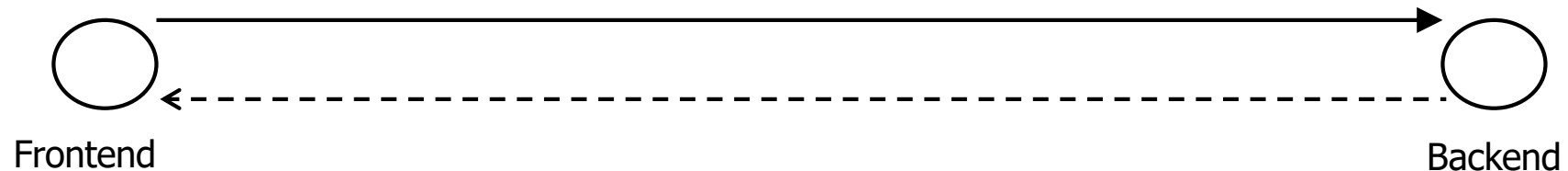
```
...
```

```
{
```

```
  "postalAddress": "Rue del Percebe, 13",
```

```
  "postalCode": "12345"
```

```
}
```



```
HTTP/1.1 404 Not Found
```

```
...
```

```
{
```

```
  "globalError": "shopping cart is empty"
```

```
}
```

Gestión de excepciones – Ejemplo 1

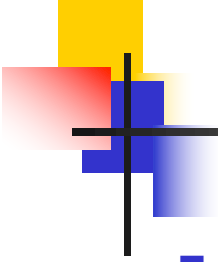
■ En ShoppingController

```
@ExceptionHandler(EmptyShoppingCartException.class) ←
@ResponseStatus(HttpStatus.NOT_FOUND)
@ResponseBody
public ErrorsDto handleEmptyShoppingCartException(
    EmptyShoppingCartException exception, Locale locale) {

    String errorMessage = ...

    return new ErrorsDto(errorMessage);
}

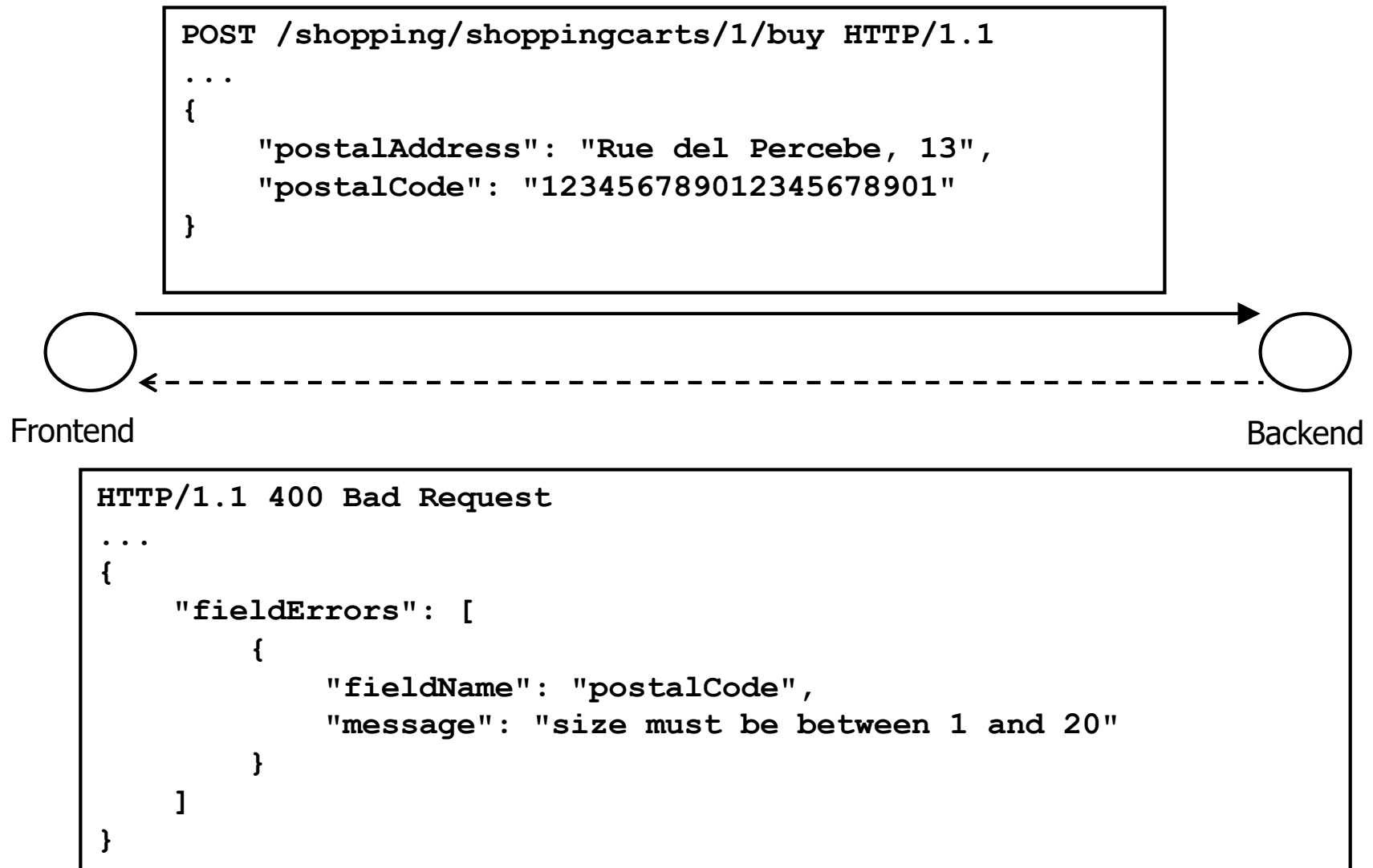
@PostMapping("/shoppingcards/{shoppingCartId}/buy")
public Long buy(@RequestAttribute Long userId,
    @PathVariable Long shoppingCartId,
    @Validated @RequestBody BuyParamsDto params)
    throws InstanceNotFoundException, PermissionException,
        < EmptyShoppingCartException > {
    ...
}
```

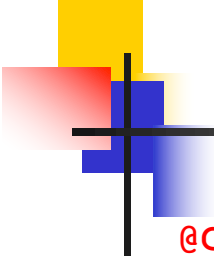


Gestión de excepciones – Ejemplo 1

- En el caso de estudio, las excepciones que son específicas a un controlador se tratan en el controlador
- Por cada excepción se define un método que la trata, anotado con
 - **@ExceptionHandler**: spring-web invocará al método cuando uno de los métodos del controlador lance la excepción especificada
 - **@ResponseStatus**: código de estado de la respuesta
 - **@ResponseBody**: la representación del valor de retorno del método se usará como contenido del cuerpo

Gestión de excepciones – Ejemplo 2





Gestión de excepciones – Ejemplo 2

`@ControllerAdvice`

```
public class CommonControllerAdvice {
```

```
...
```

```
@ExceptionHandler(MethodArgumentNotValidException.class)
```

```
@ResponseStatus(HttpStatus.BAD_REQUEST)
```

```
@ResponseBody
```

```
public ErrorsDto handleMethodArgumentNotValidException(  
    MethodArgumentNotValidException exception) {
```

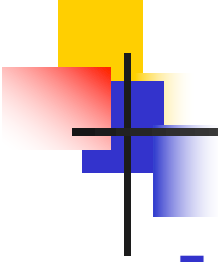
```
    List<FieldErrorDto> fieldErrors = exception.getBindingResult().  
        getFieldErrors().stream().map(  
            error -> new FieldErrorDto(error.getField(),  
            error.getDefaultMessage()))  
        .collect(Collectors.toList());
```

```
    return new ErrorsDto(fieldErrors);
```

```
}
```

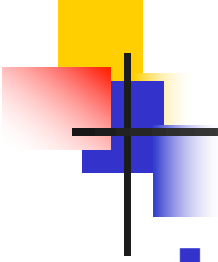
```
...
```

```
}
```



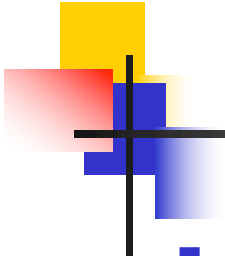
Gestión de excepciones – Ejemplo 2

- spring-web permite tratar las excepciones que son comunes a varios controladores en un única clase anotada con **@ControllerAdvice**
 - La instancia la crea Spring (es un bean), y en consecuencia se puede usar inyección de dependencias sobre ella (e.g. **@Autowired**)
- En el caso de estudio, las excepciones comunes a los controladores se tratan en **CommonControllerAdvice**
 - En el ejemplo anterior, se trata **MethodArgumentNotValidException**, que se produce cuando se incumple una regla de validación en un parámetro anotado con **@Validated**



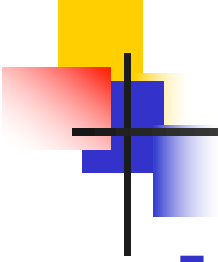
Internacionalización de mensajes

- Los dos ejemplos anteriores incluían mensajes de error en las respuestas
- Los mensajes de error deben emitirse en el idioma que prefiere el usuario
- La emisión de los mensajes en un idioma concreto se denomina internacionalización (i18n) de mensajes
- Más formalmente, la internacionalización se hace en base a un “locale”
 - Abstracción que se compone de 3 partes: idioma, país y variante
 - E.g. Véase JavaDoc de `java.util.Locale`
- En un sentido amplio, la internacionalización no trata sólo la emisión de mensajes, sino cualquier otro aspecto relativo a un locale concreto: entrada/salida de datos numéricos, fechas, cantidades monetarias, etc.



Internacionalización de mensajes

- El backend del caso de estudio sólo trata la internacionalización de los mensajes
 - El frontend tratará también otros aspectos (formateo de fechas y cantidades numéricas)
 - Por sencillez, en la base de datos, el título y descripción de los productos, así como los nombres de categorías, no están internacionalizados
- Con Spring, la internacionalización de mensajes consiste en
 - Externalizar los mensajes en ficheros properties, con formato **clave=texto del mensaje**
 - Deben estar en el classpath
 - Usar el bean de tipo **MessageSource** para recuperar un mensaje en un locale concreto a partir de su clave



Internacionalización de mensajes

- Los navegadores pueden enviar en las peticiones HTTP la cabecera **Accept-Language**
 - Lista priorizada de los locales que prefiere el usuario
 - E.g. **Accept-Language**: en
 - Algunos navegadores permiten que el usuario la defina explícitamente y otros se integran con las preferencias de locale del sistema operativo
- En el código fuente del backend del caso de estudio, los mensajes residen en **backend/src/main/resources**
 - `messages_en.properties`: mensajes en inglés
 - `messages_es.properties`: mensajes en español
 - `messages_gl.properties`: mensajes en gallego

Código de locale. Formato general: idioma[_país[_variante]]

Internacionalización de mensajes – Ejemplo 1

■ En ShoppingController

```
import org.springframework.context.MessageSource;

...

@Autowired
private MessageSource messageSource;

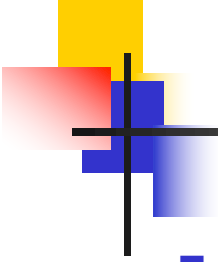
@ExceptionHandler(EmptyShoppingCartException.class)
@ResponseStatus(HttpStatus.NOT_FOUND)
@ResponseBody
public ErrorsDto handleEmptyShoppingCartException(
    EmptyShoppingCartException exception, Locale locale) {

    String errorMessage = messageSource.getMessage(
        EMPTY_SHOPPING_CART_EXCEPTION_CODE, clave
        null, EMPTY_SHOPPING_CART_EXCEPTION_CODE, locale);

    return new ErrorsDto(errorMessage);
}

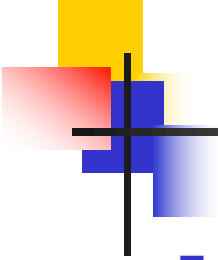
----- project.exceptions.EmptyShoppingCartException
```

clave
mensaje si clave no definida



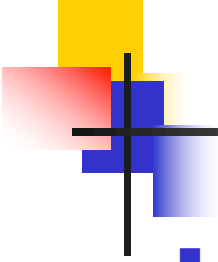
Internacionalización de mensajes – Ejemplo 1

- En los controladores, los métodos que procesan las peticiones HTTP y los gestores de excepciones pueden recibir un parámetro de tipo **Locale**
 - Spring inyecta una instancia de **Locale** en base al valor de la cabecera **Accept-Language**
- **messages_en.properties**
project.exceptions.EmptyShoppingCartException=shopping cart is empty
- **messages_es.properties**
project.exceptions.EmptyShoppingCartException=el carrito está vacío
- **messages_gl.properties**
project.exceptions.EmptyShoppingCartException=o carro está baleiro



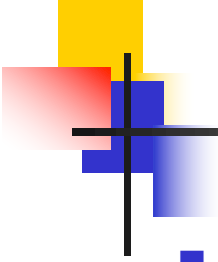
Internacionalización de mensajes – Ejemplo 2

- En algunos mensajes es necesario sustituir partes variables
- El segundo parámetro de `MessageSource.getMessage` permite pasar un vector de valores a sustituir en el mensaje
- Ejemplo: ver código fuente de `CommonControllerAdvice.handleInstanceNotFoundException`
 - `project.exceptions.InstanceNotFoundException={0}` with identifier "{1}" not found
 - `product` with identifier '123' not found



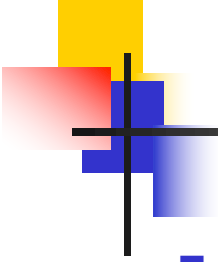
Internacionalización de mensajes – Ejemplo 3

- Mensajes de error correspondientes a las reglas de validación de Bean Validation
 - La implementación de la API de Bean Validation usada incluye los mensajes internacionalizados en inglés y español (entre otros)
 - Pero no en gallego...
- En el caso de estudio, en `messages_gl.properties` se incluyen los mensajes en gallego correspondientes a las claves de las reglas de validación usadas
 - Ejemplo: `jakarta.validation.constraints.NotNull.message=non pode ser null`
- El JavaDoc de `jakarta.validation` especifica para cada regla de validación la clave del mensaje en el campo `message`
 - Ejemplo:
<https://javadoc.io/doc/jakarta.validation/jakarta.validation-api/latest/jakarta/validation/constraints/NotNull.html> → campo `message`



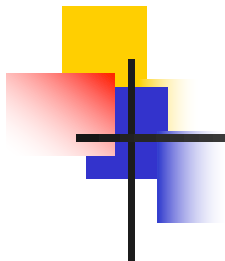
Internacionalización de mensajes – Configuración

- Es necesario crear un bean de tipo **MessageSource**
- Ver código fuente de `es.udc.pashop.backend.Application`

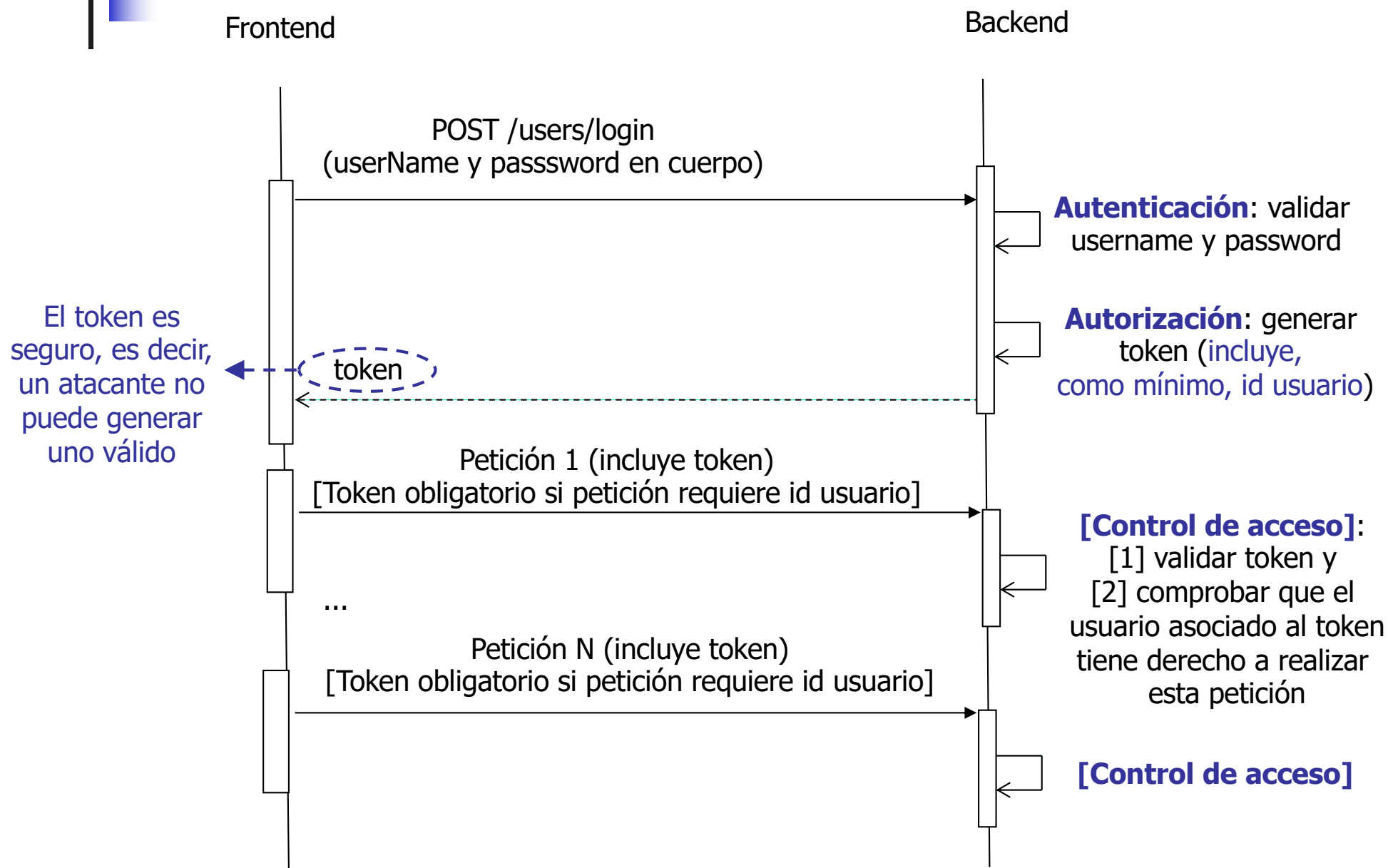


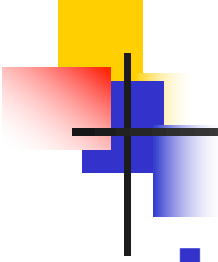
Autenticación, autorización y control de acceso

- Algunas peticiones requieren conocer el id del usuario
 - e.g.: cambiar la información de perfil, consultar los detalles de un pedido, etc.
- Es necesario emplear un esquema que permita enviar el id del usuario desde el frontend al backend de forma segura
 - E inyectar el id del usuario en el parámetro `userId` de los métodos de los controladores



Autenticación, autorización y control de acceso

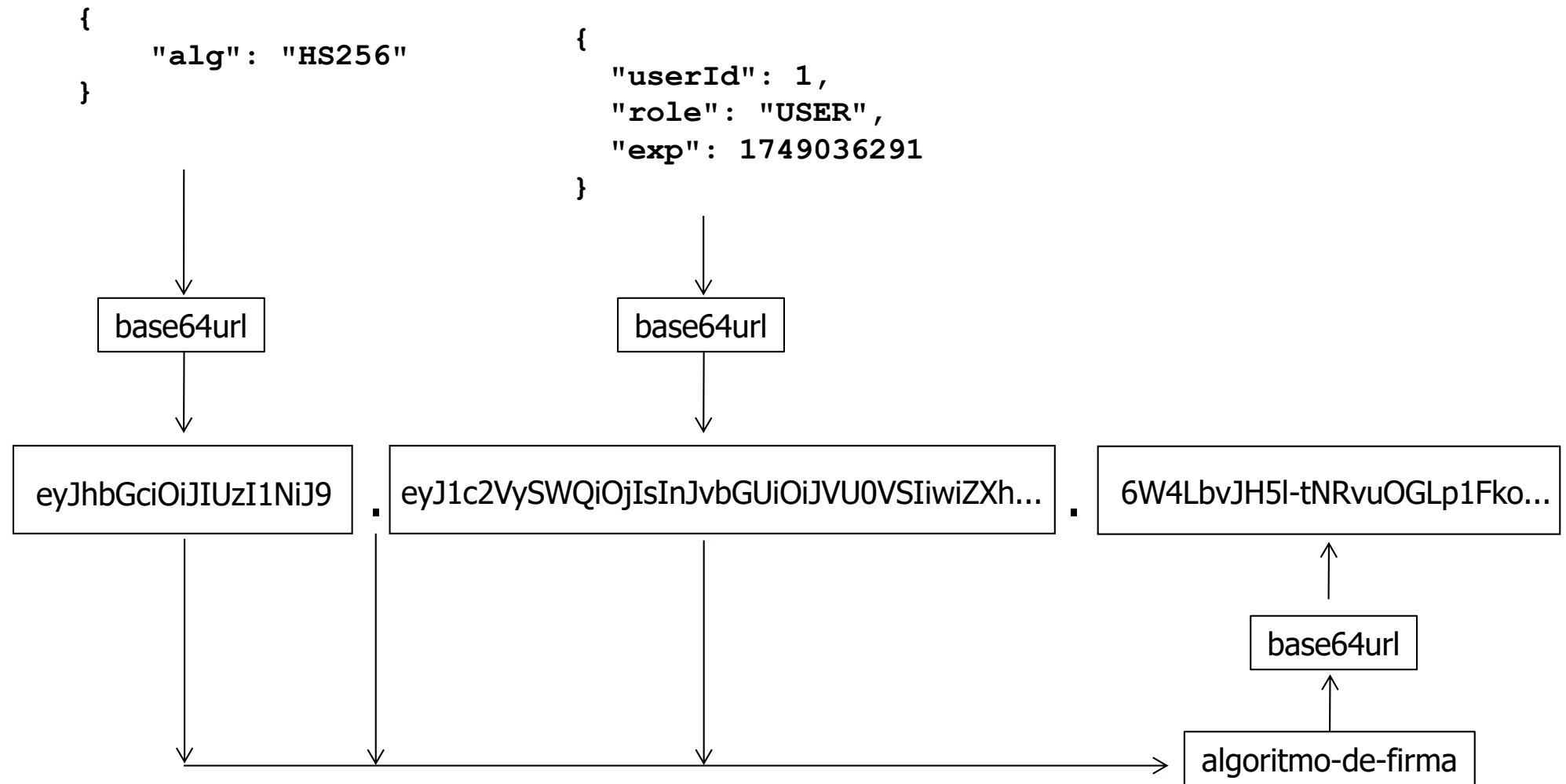




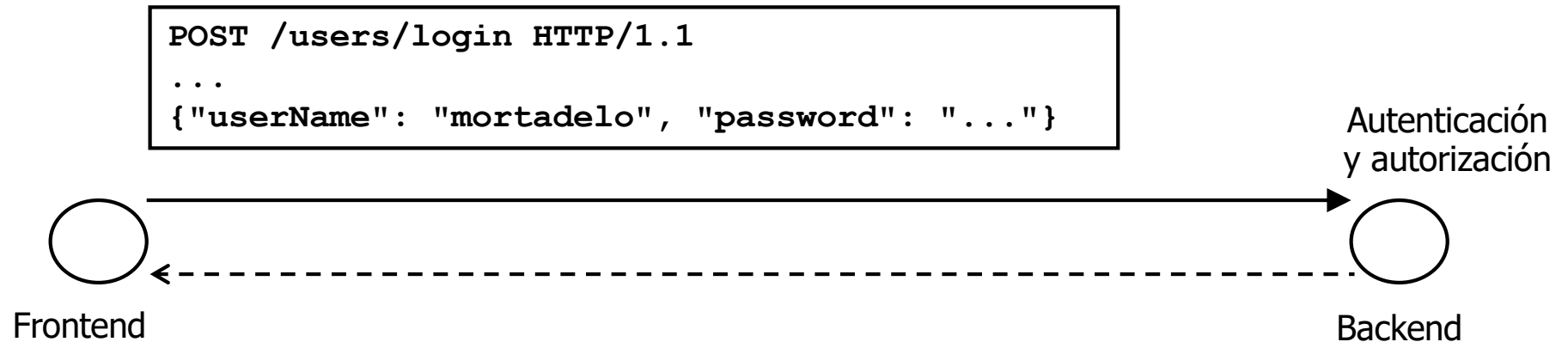
Autenticación, **autorización** y control de acceso – JWT

- Usaremos JSON Web Token (JWT) como formato de token
- JWT define un formato compacto para transmitir información en JSON de forma segura entre dos partes
 - RFC 7519
 - Sitio web interesante: <https://jwt.io>
- Formato
 - **cabecera.cuerpo.firma**
 - Cabecera
 - JSON: especifica tipo de token (JWT) y algoritmo de firma
 - Codificado en base64url
 - Cuerpo
 - JSON: contiene información ("claims") sobre una entidad (e.g. un usuario)
 - Codificado en base64url
 - Firma
 - algoritmo-de-firma(cabecera + "." + cuerpo)
 - Resultado codificado en base64url

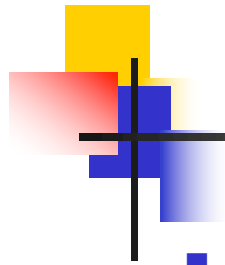
Autenticación, **autorización** y control de acceso – JWT



Autenticación, autorización y control de acceso



```
HTTP/1.1 200 OK
...
{
  "serviceToken": "eyJhbGciOiJIUzI1NiJ9.eyJ1c2VySWQiOiJEsInJvbmVudGUiOiJVU0VSIlwiZXhwIjozNzQ5MDM2MjkxZQ.6W4LbvJH51-tNRvuOGLp1FkoBdE13-dDxAJN9nND51E",
  "user": {
    "id": 1,
    "userName": "mortadelo",
    "firstName": "Mortadelo",
    ...
  },
  "shoppingCart": { ... }
}
```



Autenticación, autorización y control de acceso

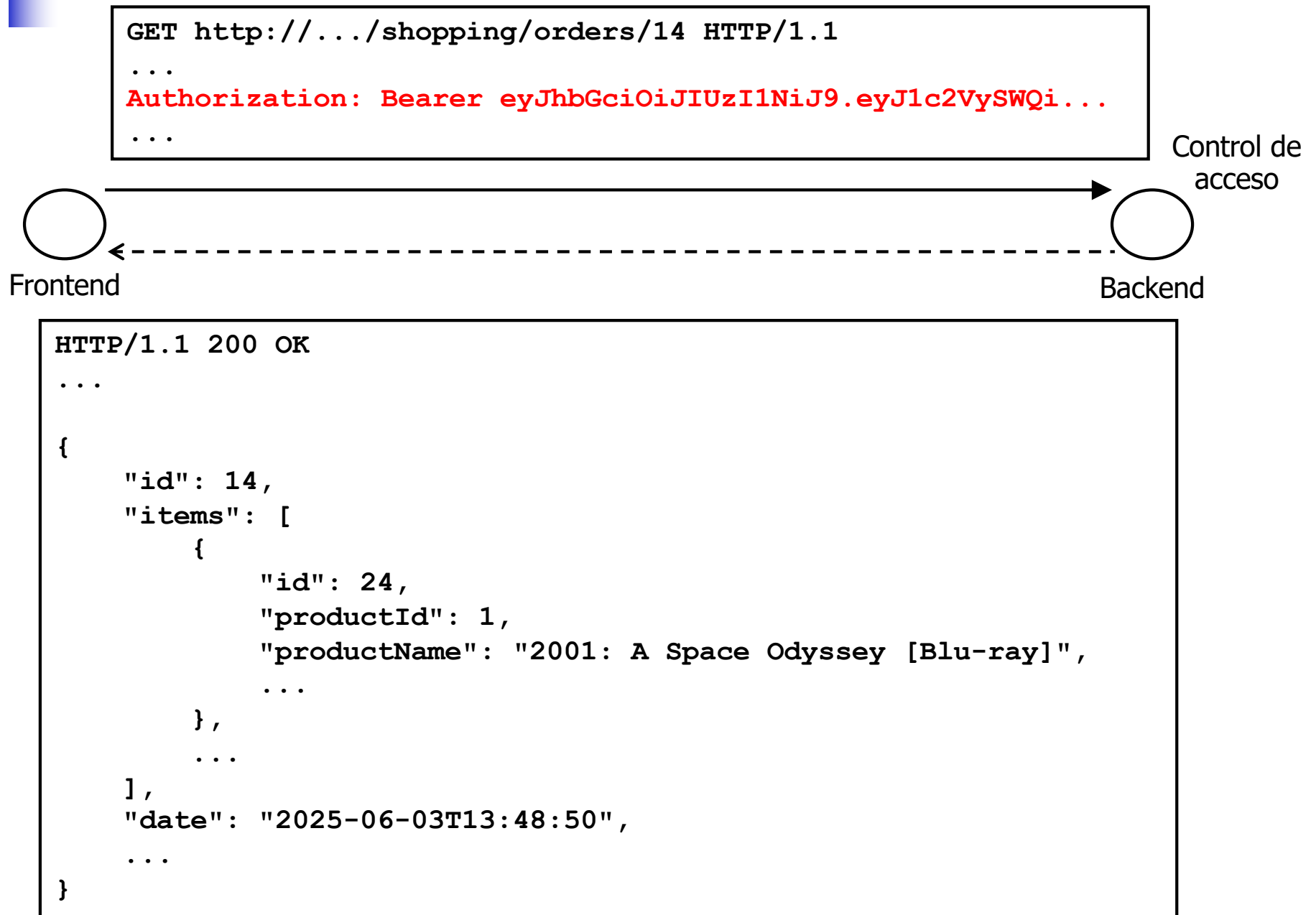
■ Autenticación

- Se recupera `user` a partir de `userName`
- Se cifra `password` y se comprueba si coincide con el valor de `user.getPassword()`

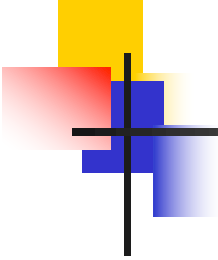
■ Autorización

- Se genera un JWT firmado con un algoritmo de criptografía simétrica (HS256)
- Se incluye en el cuerpo del token
 - `userId: user.getId()`
 - `role: user.getRole()`
 - En el caso de estudio, todos los usuarios tienen el rol `USER`, pero en una versión más completa podría existir también el rol `ADMIN`
 - `exp: fecha de expiración del token en segundos (desde 1-1-1970)`
 - En el caso de estudio los tokens se emiten con una validez de 1 día
- `POST /users/signUp` también devuelve el token

Autenticación, autorización y control de acceso



- En el caso de estudio, el control de acceso se realiza con Spring Security
 - <https://spring.io/projects/spring-security>
- Permite realizar el control de acceso de forma declarativa mediante ficheros de configuración, anotaciones o código



Autenticación, autorización y **control de acceso**

- En `SecurityConfig.securityFilterChain`

[1] `JwtFilter` (proporcionado por el caso de estudio):

- Validar token (si llega): comprobar firma válida + token no caducado
- Leer `userId` y habilitar inyección del parámetro `userId` en los métodos de los controladores
- Leer `rol` e informar a Spring Security del rol del usuario

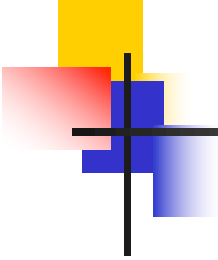
http...

```
.addFilterBefore(new JwtFilter(jwtGenerator), ...)
```

```
.authorizeHttpRequests((authorize) -> authorize
```

```
    .requestMatchers(HttpMethod.POST, "/users/signUp").permitAll()  
    .requestMatchers(HttpMethod.POST, "/users/login").permitAll()  
    .requestMatchers(HttpMethod.POST, "/users/loginFromServiceToken")  
        .permitAll()  
    .requestMatchers(HttpMethod.GET, "/catalog/categories").permitAll()  
    .requestMatchers(HttpMethod.GET, "/catalog/products/*").permitAll()  
    .requestMatchers(HttpMethod.GET, "/catalog/products").permitAll()
```

[2] Comprobar que el usuario que realiza la petición tiene los roles suficientes
(continúa en la siguiente diapositiva)



Autenticación, autorización y **control de acceso**

- En `SecurityConfig.securityFilterChain`

```
.requestMatchers(HttpMethod.PUT, "/users/*").hasRole("USER")
.requestMatchers(HttpMethod.POST, "/users/*/changePassword").hasRole("USER")
.requestMatchers(HttpMethod.POST,
    "/shopping/shoppingcards/*/addToShoppingCart").hasRole("USER")
.requestMatchers(HttpMethod.POST,
    "/shopping/shoppingcards/*/updateShoppingCartItemQuantity").hasRole("USER")
.requestMatchers(HttpMethod.POST,
    "/shopping/shoppingcards/*/removeShoppingCartItem").hasRole("USER")
.requestMatchers(HttpMethod.POST,
    "/shopping/shoppingcards/*/buy").hasRole("USER")
.requestMatchers(HttpMethod.GET, "/shopping/orders/*").hasRole("USER")
.requestMatchers(HttpMethod.GET, "/shopping/orders").hasRole("USER")
.anyRequest().denyAll();
```

[2] Comprobar que el usuario que realiza la petición tiene los roles suficientes

- No todo el control de acceso se puede realizar declarativamente en base a roles
- Ejemplo
 - Para invocar `GET /shopping/orders/{orderId}` es necesario tener el rol `USER`
 - Pero además, en la capa Lógica de Negocio, `ShoppingServiceImpl.findOrder(userId, orderId)` comprueba que el pedido pertenece al usuario